

MEMORIA DEL PROYECTO: TASKMASTER ALS

Ismael Conde Conde

05/2026

Índice

1. INTRODUCCIÓN.....	1
2. REQUISITOS TÉCNICOS Y TECNOLOGÍAS.....	1
3. DISEÑO Y ARQUITECTURA.....	1
3.1. Modelo de Datos (Entidades).....	1
4. FUNCIONALIDADES IMPLEMENTADAS.....	2
4.1. Gestión de Usuarios.....	2
4.2. CRUD Completo.....	2
4.3. Navegación e Interfaz.....	3
5. ALMACENAMIENTO E INTEGRIDAD ESTRUCTURAL.....	3
6. EJECUCIÓN DEL PROYECTO.....	3
7. CONCLUSIÓN.....	4

1. INTRODUCCIÓN

El proyecto “TaskMaster ALS” es una aplicación web de gestión de productividad desarrollada sobre el micro-framework Flask. El objetivo principal es permitir a los usuarios organizar su carga de trabajo mediante una jerarquía de tres niveles: Categorías, Proyectos y Tareas.

La aplicación pone especial énfasis en la experiencia de usuario (UX), la seguridad mediante sesiones autenticadas y la integridad estructural de los datos almacenados en una base de datos NoSQL.

2. REQUISITOS TÉCNICOS Y TECNOLOGÍAS

Para el desarrollo de la práctica se han empleado las siguientes tecnologías exigidas:

- Lenguaje: Python.
- Framework Web: Flask.
- Motor de Plantillas: Jinja2.
- Persistencia de Datos: Redis mediante la biblioteca Sirope, empleando un modelo de persistencia de objetos.
- Gestión de Usuarios: Flask-Login para el manejo de sesiones y Werkzeug para el hashing de contraseñas.
- Front-end: HTML5, CSS3 y el framework minimalista Pico CSS.

3. DISEÑO Y ARQUITECTURA

La aplicación sigue un diseño modular, separando claramente las responsabilidades:

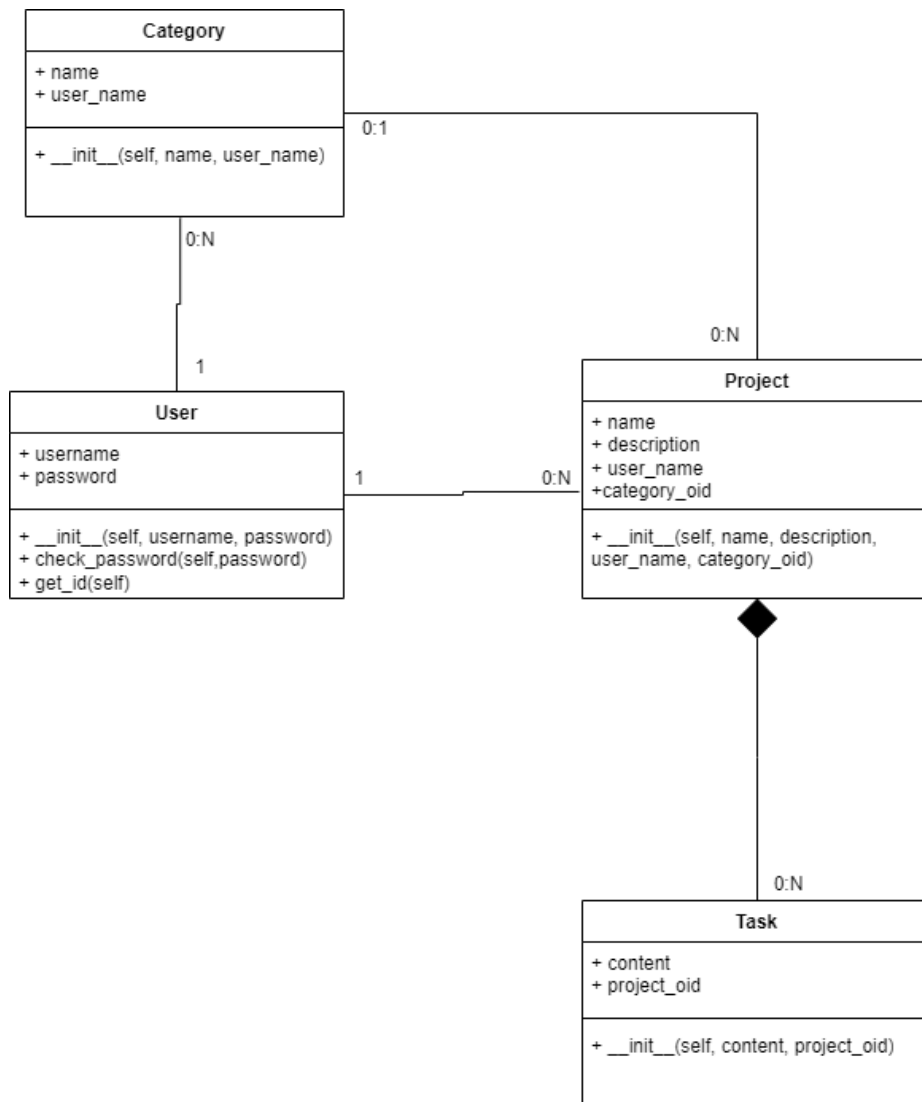
- `model.py`: Contiene la definición de las clases (entidades).
- `app.py`: Actúa como controlador principal, gestionando las rutas, la lógica de negocio y la comunicación con la capa de persistencia.
- `templates/`: Capa de presentación que utiliza Jinja2 para renderizar datos dinámicos

3.1. Modelo de Datos (Entidades)

Se han implementado tres entidades adicionales a la gestión de usuarios:

1. Categoría (Category): Permite agrupar proyectos (ej: “Trabajo”, “Universidad”).

2. Proyecto (Project): Entidad central que contiene una descripción y un estado global (Completado/Pendiente).
3. Tarea (Task): La unidad mínima de trabajo, vinculada a un proyecto específico.



4. FUNCIONALIDADES IMPLEMENTADAS

4.1. Gestión de Usuarios

Se ha implementado un sistema completo de registro e inicio de sesión. Las contraseñas no se almacenan en texto plano, sino mediante el uso de hashes criptográficos (pbkdf2:sha256), garantizando la seguridad de las credenciales.

4.2. CRUD Completo

La aplicación permite realizar todas las operaciones básicas (Create, Read, Update, Delete) sobre

las tres entidades:

- Creación y eliminación de categorías.
- Creación, edición y borrado de proyectos.
- Añadido, edición de contenido y eliminación de tareas.
- Marcado dinámico de tareas y proyectos como "Completados".

4.3. Navegación e Interfaz

Se ha diseñado una interfaz intuitiva utilizando Pico CSS. El sistema de navegación permite desplazarse entre la lista de proyectos, la gestión de categorías y el detalle de cada proyecto sin necesidad de introducir URLs manualmente. Se han incluido diálogos de confirmación en JavaScript para operaciones críticas (borrado) para mejorar la robustez.

5. ALMACENAMIENTO E INTEGRIDAD ESTRUCTURAL

El uso de Sirope ha permitido gestionar los objetos en Redis de forma eficiente. Se han implementado mecanismos para garantizar la consistencia de los datos:

- Borrado en Cascada: Al eliminar un objeto de la entidad Project, el sistema identifica y elimina automáticamente todas las Task asociadas a dicho proyecto. Esto evita la existencia de "objetos huérfanos" en la base de datos.
- Gestión de Referencias: Al eliminar una Category, los proyectos asociados actualizan su referencia a None, permitiendo que el sistema siga funcionando bajo la etiqueta "Sin categoría".
- Robustez ante datos antiguos: Se han empleado métodos como getattr() y parches de inicialización en las rutas para asegurar que objetos creados en versiones anteriores del modelo no provoquen excepciones de tipo AttributeError.

6. EJECUCIÓN DEL PROYECTO

La aplicación está preparada para ser ejecutada en un entorno local mediante los siguientes comandos:

1. Inicio del servidor de datos: redis-server
2. Inicio de la aplicación web: flask run (o python app.py)

El acceso se realiza a través de la dirección: <http://127.0.0.1:5000>

7. CONCLUSIÓN

El proyecto cumple satisfactoriamente con todos los requisitos exigidos. La combinación de Flask y Sirope ha permitido crear una herramienta ágil, robusta y con una arquitectura de datos sólida, demostrando la capacidad de gestionar múltiples entidades relacionadas y garantizando la integridad de la información en todo momento.